

Dana Monzer – IEMS 469 – HW#3 – Part 2

Noise for all pixels

Note that the image used as a sample in all this report refers to the same image, which is the 57th image of client 68. Also, note that the numbering of the runs is only helpful to track which folders correspond to which runs. Some runs were not included because the results weren't adequate (for example the hyperparameters of the model weren't good to start with).

The Laplace mechanism perturbs the input data by injecting random noises drawn from the Laplace distribution. Specifically, for a 28×28 image X in the Federated EMNIST dataset, a Laplace noise of scale b is added to each pixel of the image to get the perturbed image X' , i.e.,

$$X' = X + \varepsilon,$$

where ε is a 28×28 matrix with each entry ε_{ij} sampled from a 0-centered Laplace distribution with scale b . The Laplace distribution centered at 0 with scale b has the following probability density function.

$$\text{Lap}(x|b) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right)$$

To implement Laplace, I use `numpy.random.laplace` function in `numpy` module. Refer to the following link:

<https://numpy.org/doc/stable/reference/random/generated/numpy.random.laplace.html>

The function is equivalent to what we want to implement, except that we need to set $\mu = 0$.

$$f(x; \mu, \lambda) = \frac{1}{2\lambda} \exp\left(-\frac{|x - \mu|}{\lambda}\right).$$

1 Training on perturbed dataset with different noise scales

Use images perturbed by Laplace noises. Plot train/validation accuracies for different noise scales of your choice and show proper learning given the noisy inputs. Report test accuracy.

A. Run2

For this run, I used the following hyperparameters:

- `batch_size = 32`
- `initial_learning_rate = 0.00001`
- `global_model = get_model(initial_learning_rate)`
- `total_rounds = 200`
- `C = 10`
- `E = 30`

B values: [0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.2, 0.3, 0.5, 0.8, 1, 2, 3, 4, 5]

The test accuracies for the following scenarios of b-values are listed in the following array:

```
[0.5098039507865906,  
0.514498770236969,  
0.5211267471313477,  
0.5280309319496155,  
0.5131179094314575,  
0.4998619258403778,  
0.49295774102211,  
0.49682408571243286,  
0.4722452461719513,  
0.4043081998825073,  
0.21817177534103394,  
0.10439105331897736,  
0.06241369619965553,  
0.055509526282548904,  
0.030102182179689407,  
0.06075669825077057,  
0.022921845316886902]
```

The noisy images for client 68's 57th image are in folder Part 2\Run 2\noisy image.

Below, we show a few examples of noisy images corresponding to a certain scale and the training and validation accuracy of perturbed images. For the other values of b, refer to folder: \Part 2\Run 2\accuracy plots.

Note that the original image looks like this:

B=0.03

Figure 1 shows an example of perturbed image with this scale. Figure 2 shows that the validation accuracy is increasing close to the training accuracy and they both converge close to 0.6, so the model is still able to learn and generalize the model to other datasets, and the noise has very little effect (this result is comparable to the results in Part 1).

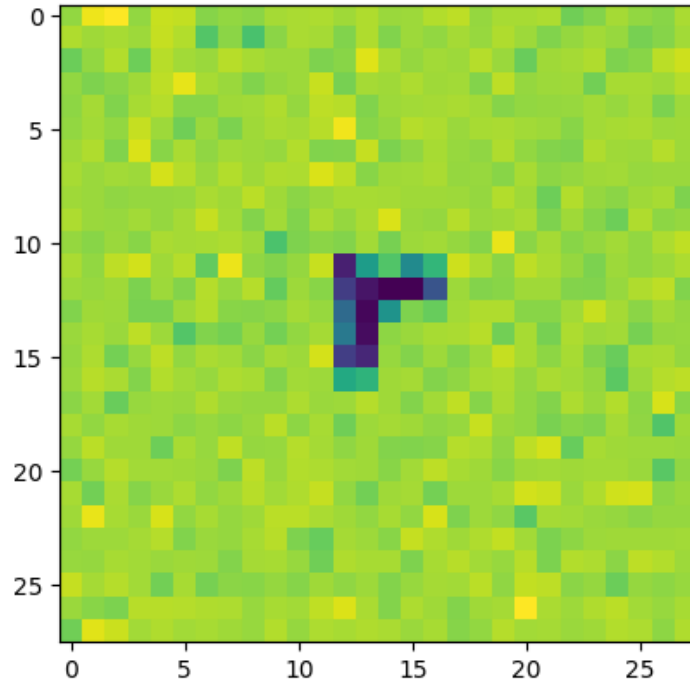


Figure 1: Perturbed Image with noise scale = 0.03

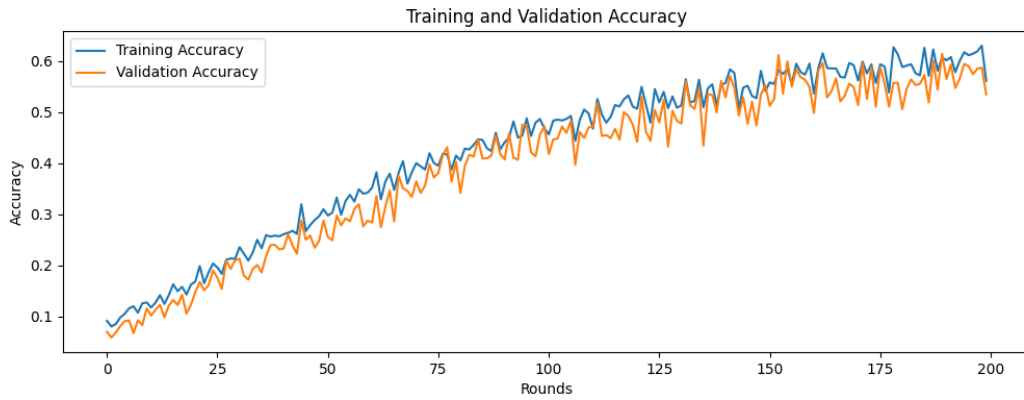


Figure 2: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 0.03

B=0.1

Figure 4 shows an example of perturbed image with this scale. Figure 3 shows that the validation accuracy is increasing close to the training accuracy and converging, so the model is still able to learn and generalize the model to other datasets. However, the learning quality deteriorates slightly from Figure 2 with only scale =0.01.

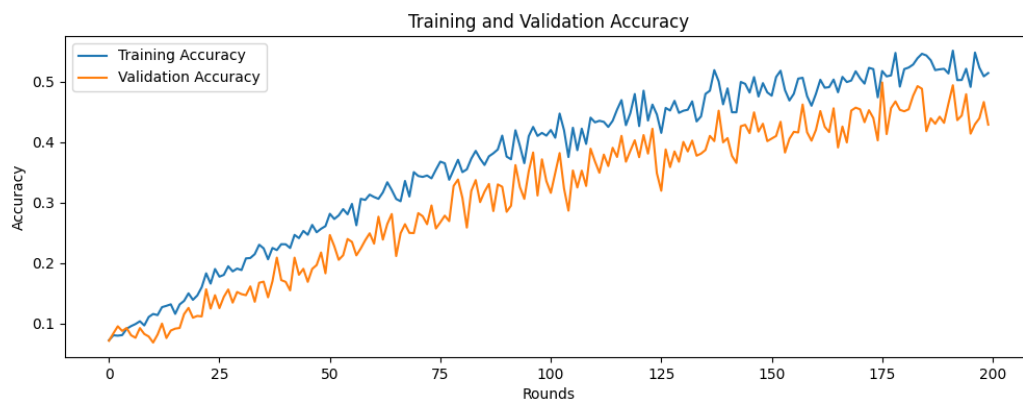


Figure 3: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 0.1

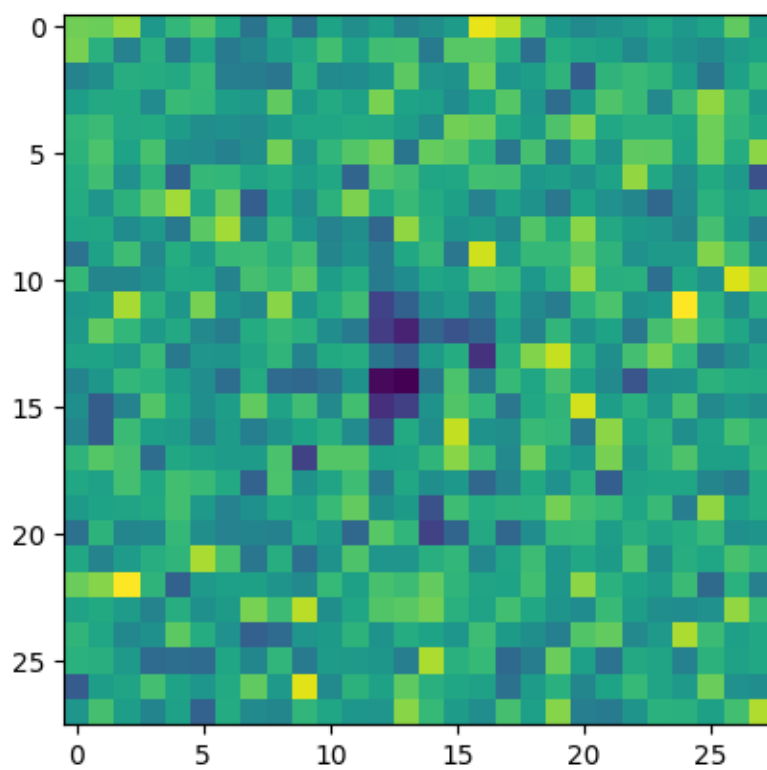


Figure 4: Perturbed Image with noise scale = 0.1

B=0.5

Figure 5 shows an example of perturbed image with this scale. Figure 6 shows that the validation accuracy is stuck at a low value, while the training accuracy is increasing but also gets stuck at a low value, this shows that the model is not able to learn much.

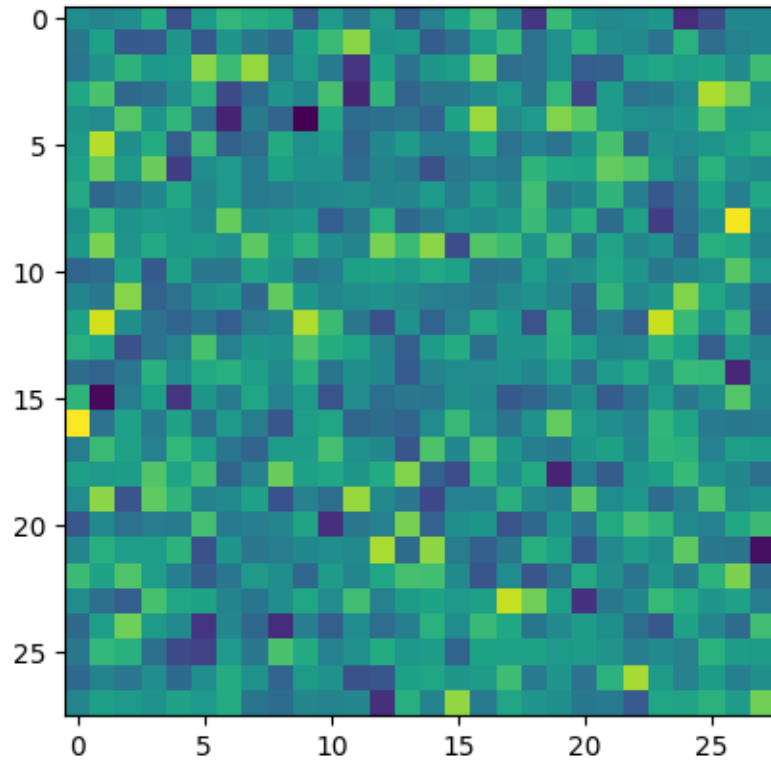


Figure 5: Perturbed Image with noise scale = 0.5

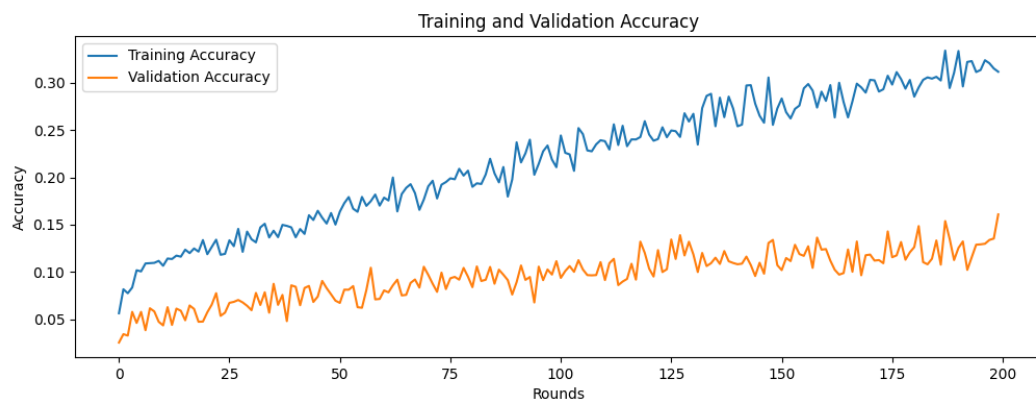


Figure 6: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 0.5

B=3

Figure 7 shows an example of perturbed image with this scale. Figure 8 shows that the validation accuracy is very low, while the training accuracy is increasing, this shows that the model is trying to overfit the data, without learning to generalize the model, so it is probably learning a wrong model.

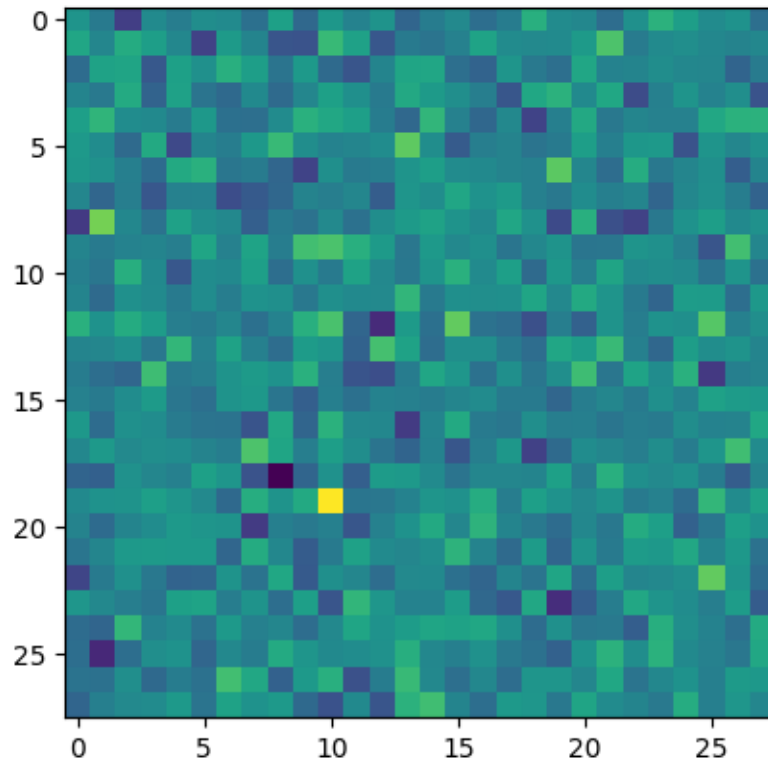


Figure 7: Perturbed Image with noise scale = 3

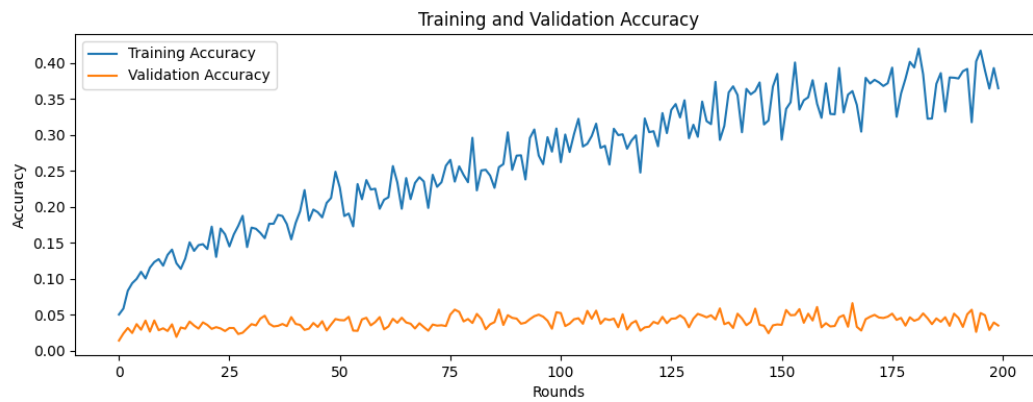


Figure 8: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 3

B. Run 6

The hyperparameters of this run:

- `batch_size` = 32
- `initial_learning_rate` = 0.00001
- `total_rounds` = 500
- `C` = 10
- `E` = 30

B= 0.08

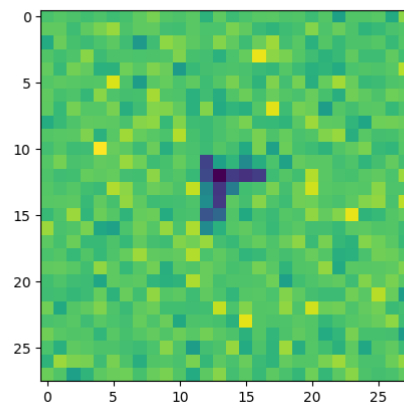


Figure 9: Perturbed Image with noise scale = 0.08

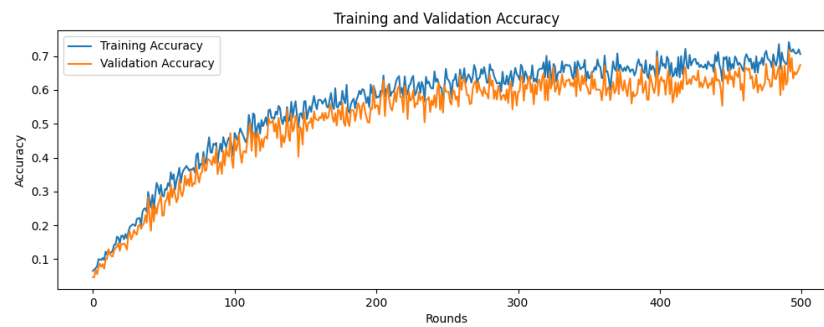


Figure 10: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 0.08

B=0.1

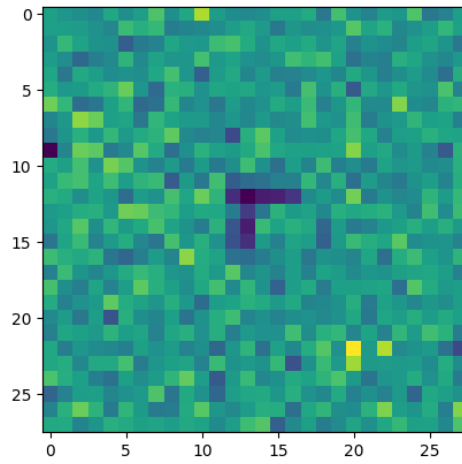


Figure 11: Perturbed Image with noise scale = 0.1

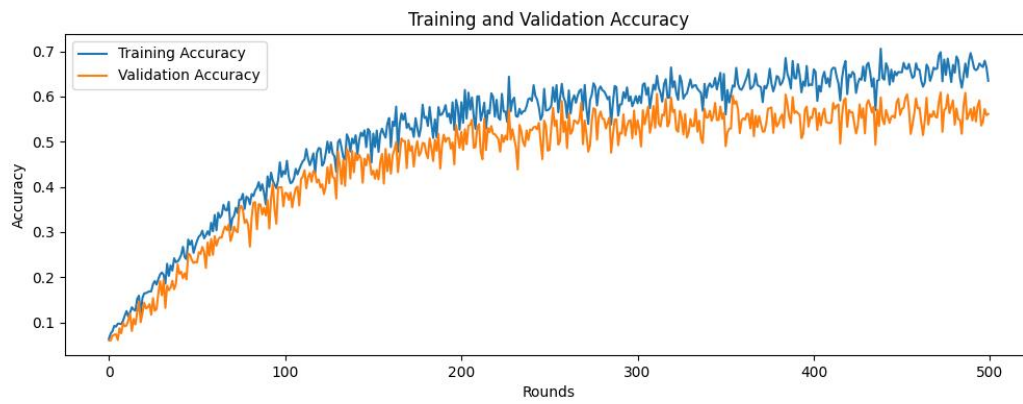


Figure 12: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 0.1

B=0.4

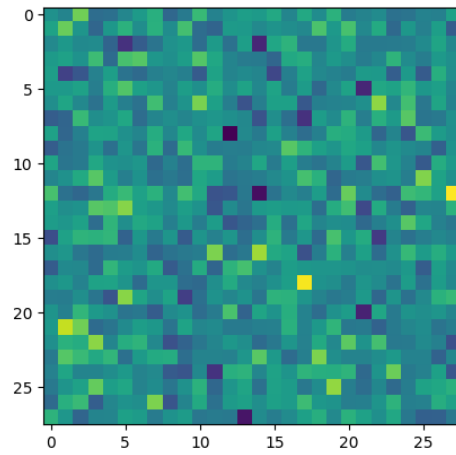


Figure 13: Perturbed Image with noise scale = 0.4

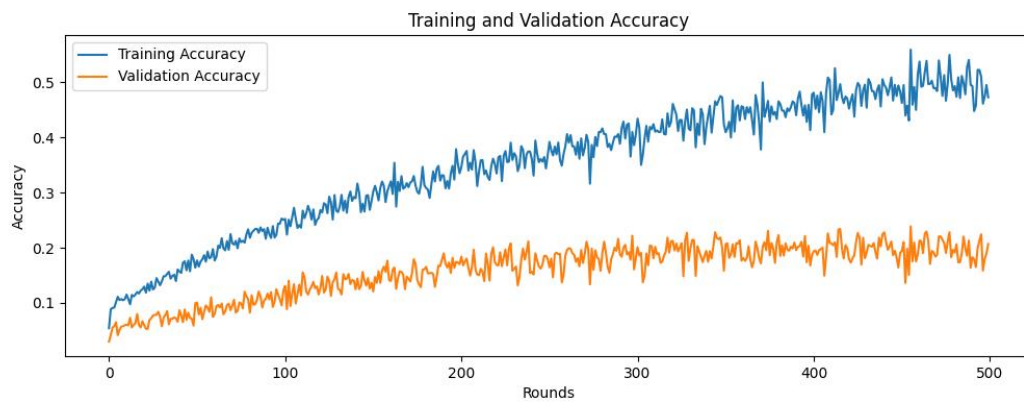


Figure 14: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 0.4

B=1.1

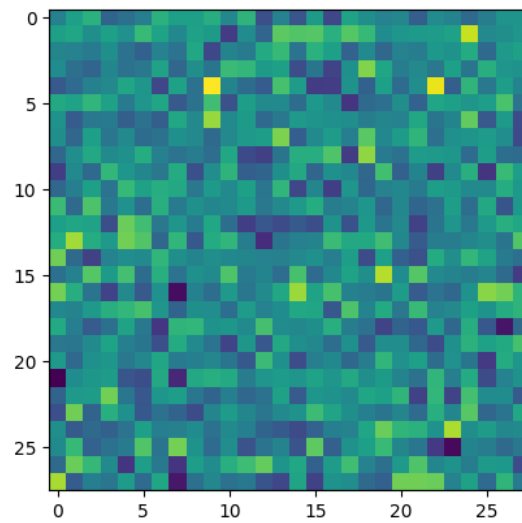


Figure 15: Perturbed Image with noise scale = 1.1

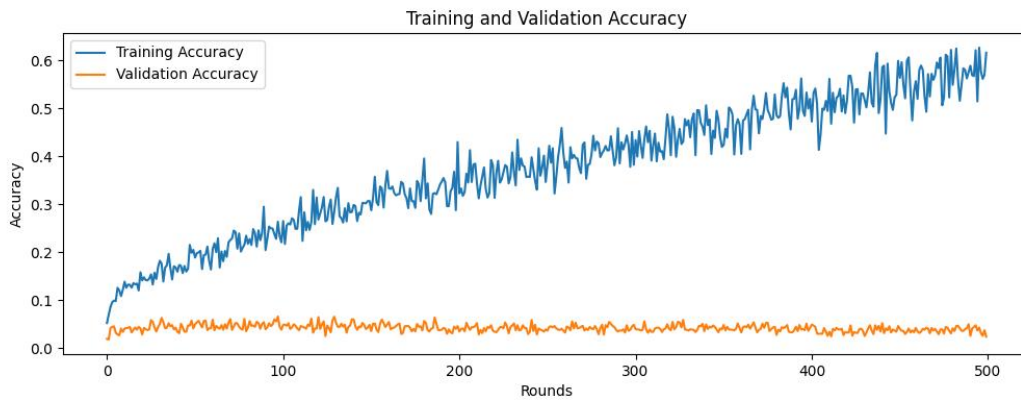


Figure 16: Aggregated Training and Validation accuracy of the data perturbed by noise scale = 1.1

2- Plot of accuracy vs noise scale

Plot of final training accuracy and test accuracy that accurately demonstrates the relationship between the noise scale and the learning quality.

A. Run 2

All the validation accuracy reaches a plateau (asymptote at around 200 rounds) or is stuck at certain values at around 200 rounds, which shows that the learning has converged. So we can compare the accuracies of different scales.

Figure 9 shows the curves of training, validation, and test accuracies as a function of noise scale. Figure 10 shows the bar plot of test accuracy across different scales. And Figure 11 shows the bar plot of training, validation, and test accuracies for different scale values. We notice that the validation and test accuracies drop somewhat logarithmically as the noise scale increases. And they reach a plateau after scale =1. The training accuracy on the other hand seem to decrease and then increase, but since the validation and test accuracies are low, the learning quality drops after scale =1 to become very poor.

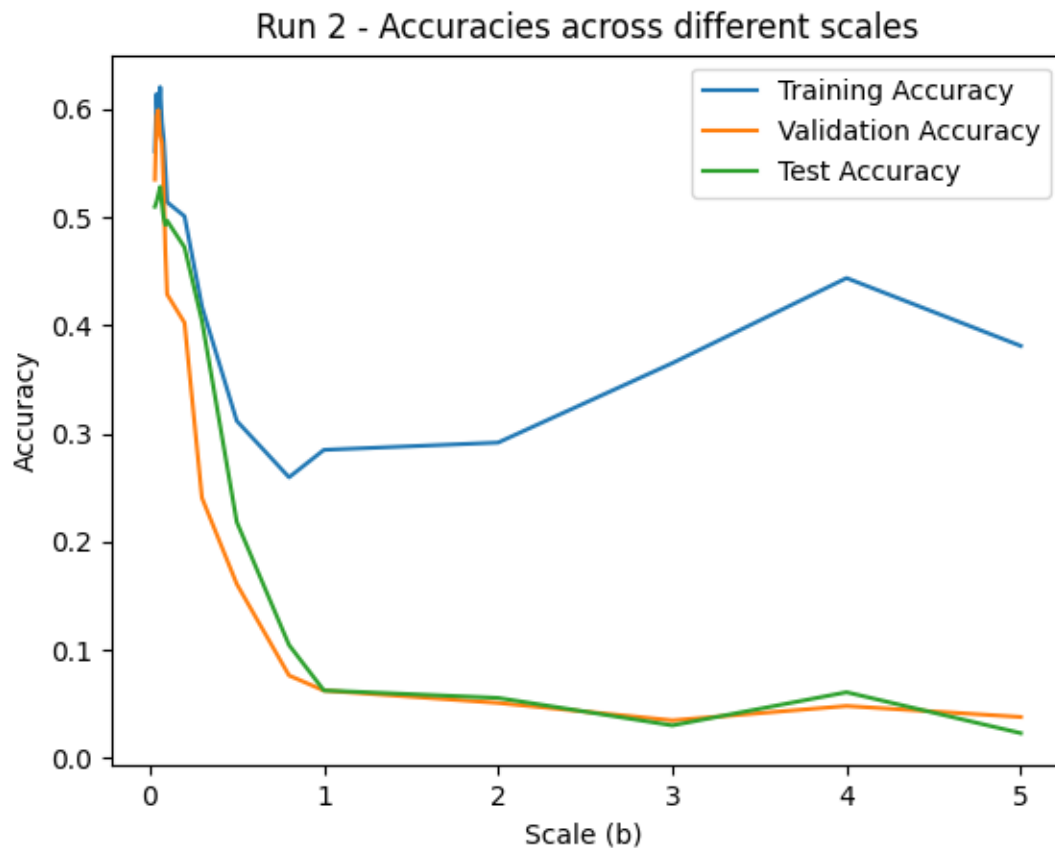


Figure 17: The curves showing the accuracy values across different noise scale values

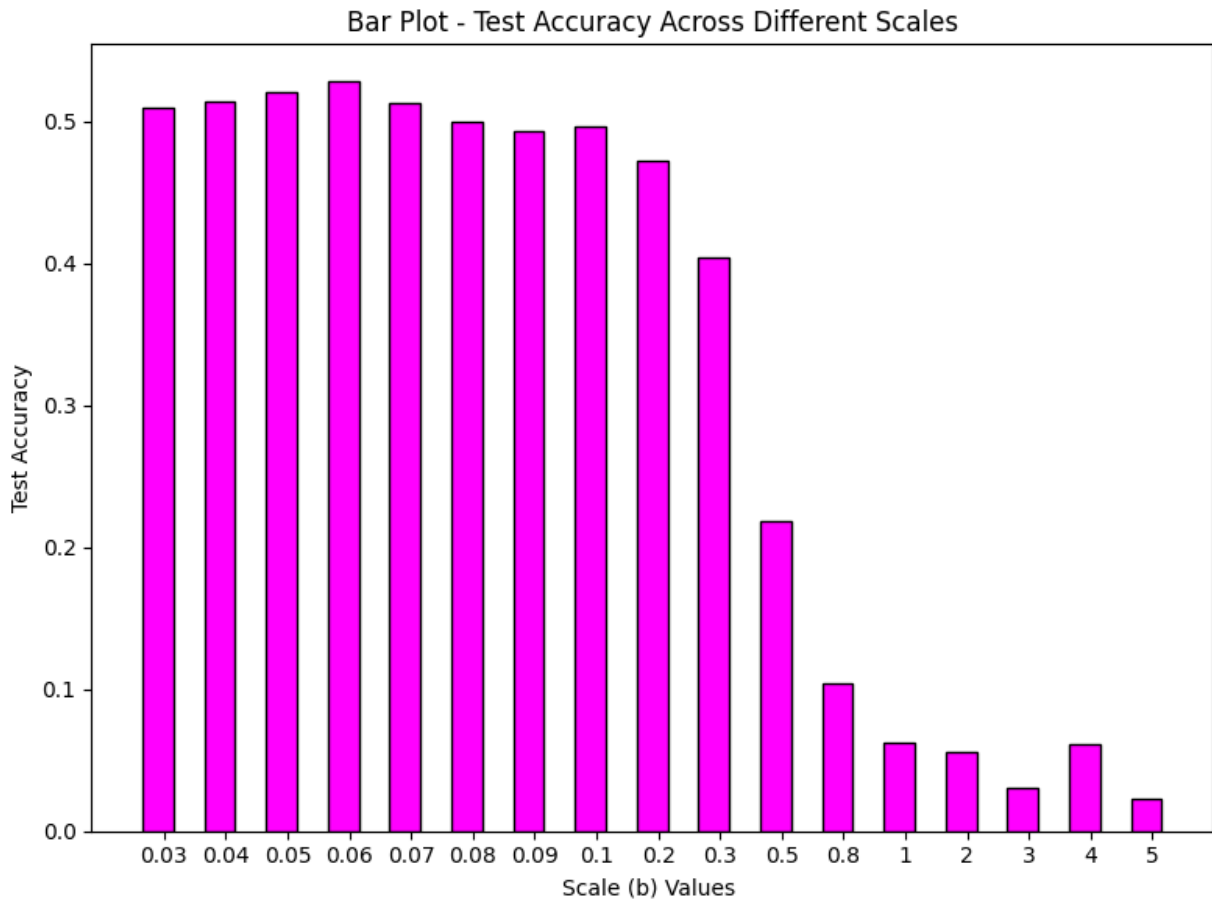


Figure 18: Bar Plot of the Test Accuracy across different scale values

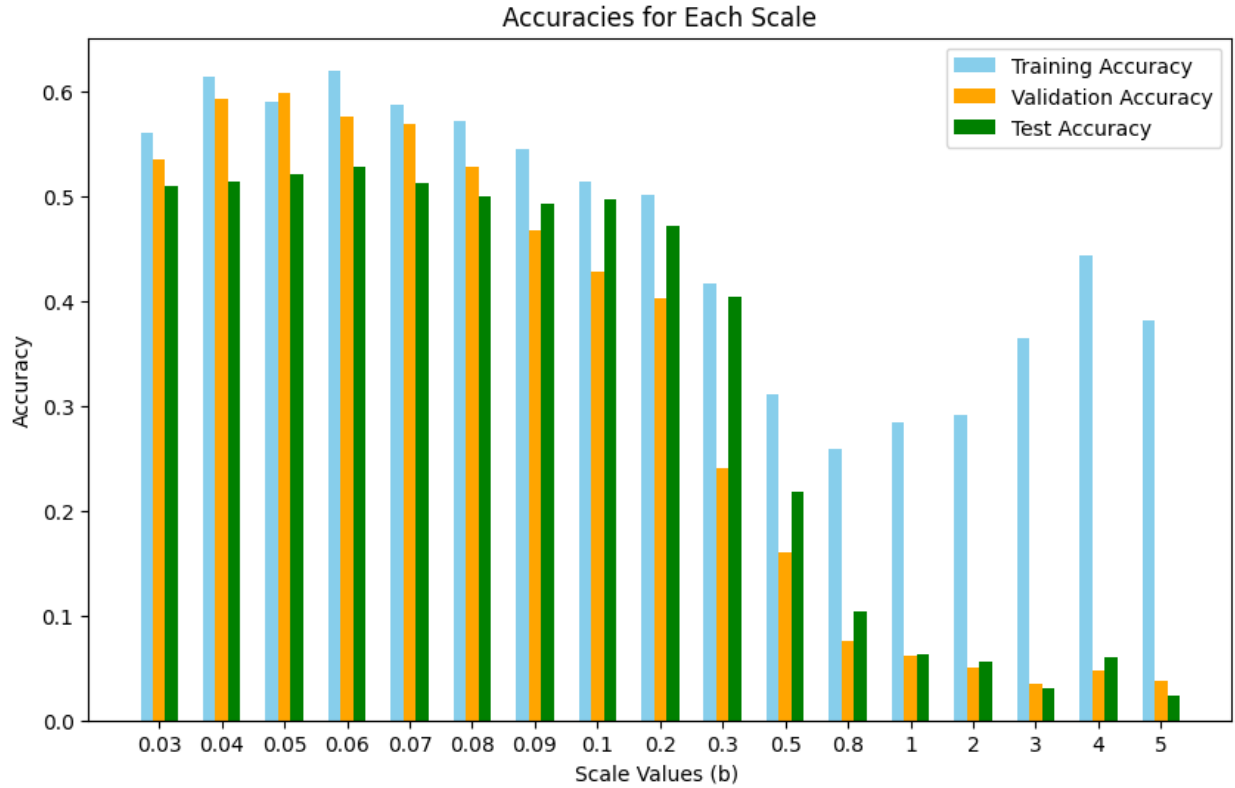


Figure 19: Bar Plot of the Accuracy values across different scale values for Run 2

B. Run 6

Figure 20 shows that both validation and test accuracy values decrease as the scale of the noise in the Laplace distribution increases. However, the training accuracy decreases a little then increases again and it stays higher than 0.6 approximately. The possible explanation for this is that the model when training tries to memorize the model and overfit the data, however, fails to generalize the model to other new datasets.

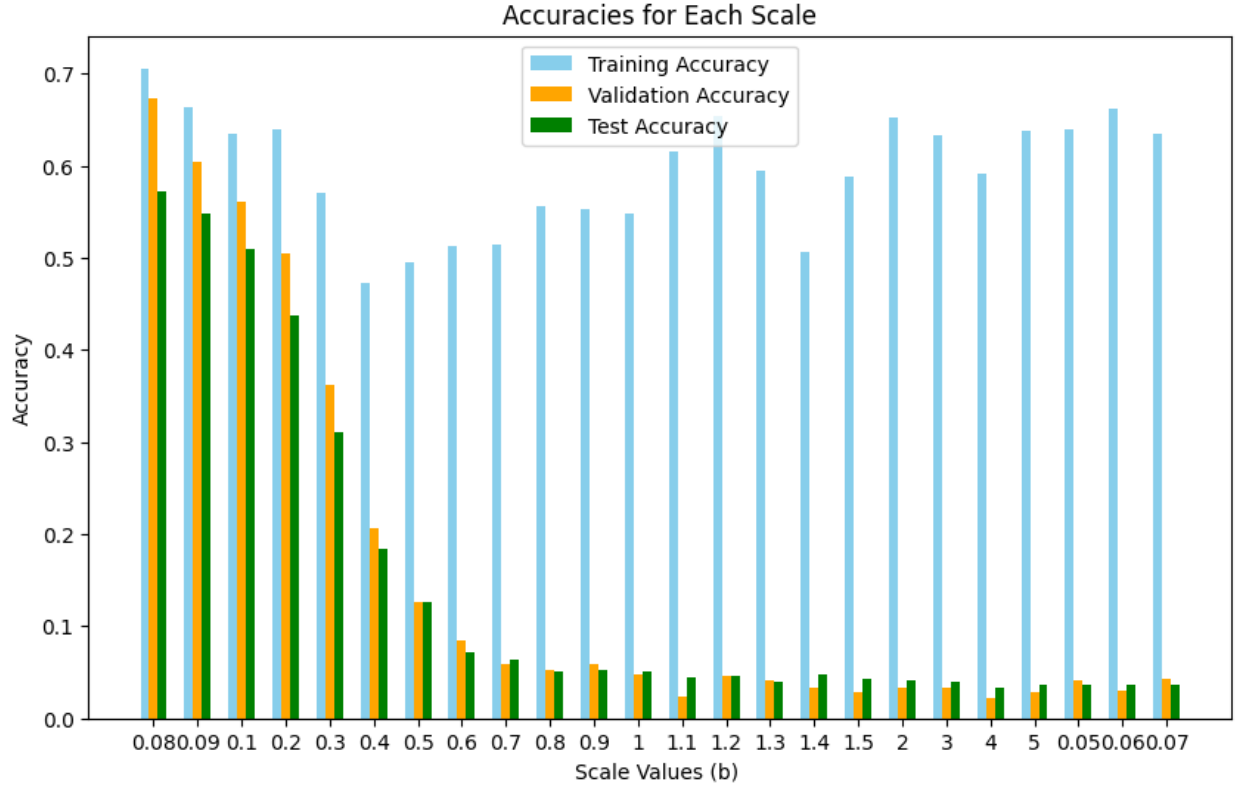


Figure 20: Bar Plot of the Accuracy values across different scale values for Run 6

C. Run 1

This run was only tested on low values of scale values, because up to $b=0.1$ only, we can guarantee the Laplace distribution will produce no values higher than 1. And the pixels of the data are normalized, and noise clipping is not recommended. However, after running simple trials, it seems that having a perturbed pixel higher than 1 is not a problem and is still translated to an image appropriately. So, this run was limited to values less than $b=0.1$.

The figure in Figure 21 shows that all scale values between 0.0001 and 0.03 have almost the same training, validation, and test accuracies. This shows that we can safely increase the noise scale at least up to 0.03 without decreasing the learning quality, almost at all.

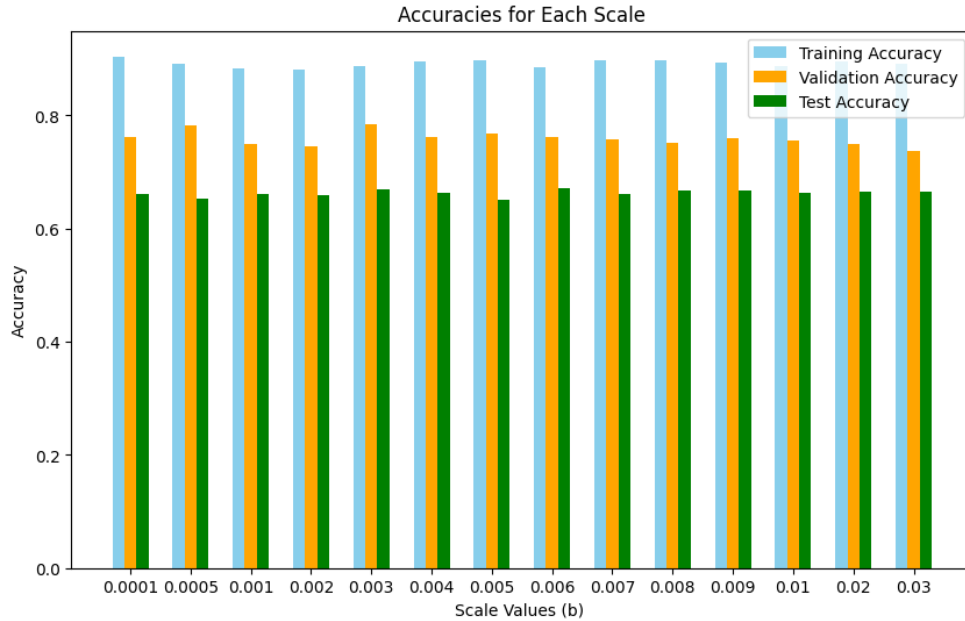


Figure 21: Bar Plot of the Accuracy values across different scale values for Run 1

3- Relationship between the noise scale and the learning quality

Description of the relationship with reasonable interpretation. Brief justification for your choice of b.

Differential privacy involves incorporating controlled randomization or noise into data, safeguarding individual privacy while enabling analysis of sensitive datasets. This ensures that the inclusion or exclusion of any individual's data has minimal impact on the overall outcome of a query. This makes it statistically challenging to discern if specific individuals' data was utilized in the analysis. By offering a mathematically assured level of privacy, differential privacy facilitates deriving valuable insights from sensitive datasets while reducing the risk of revealing personal information. It is in particular useful across domains like healthcare, finance, and social sciences, allowing for analyses on sensitive data while upholding individual privacy. However, when adding noise the learning quality is expected to decline. For this reason, we need to study the appropriate level of noise that preserves an acceptable level of learning quality while maintaining an acceptable level of privacy. In this assignment, we use Laplace distribution to induce noise. And for each pixel in the image we draw a random

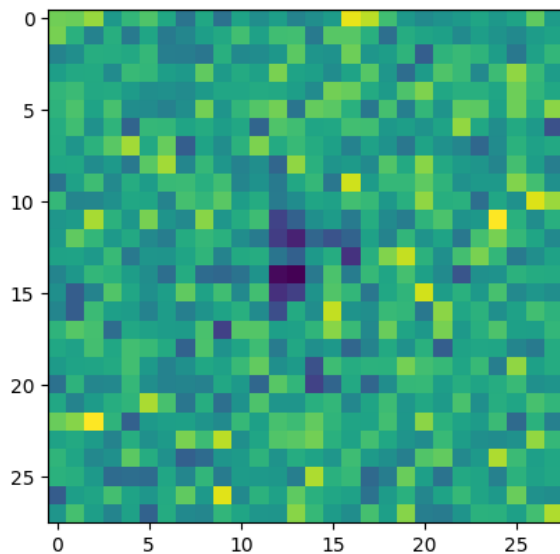
The relationship between the noise scale 'b' in a Laplace distribution and the validation and test accuracy of a model trained on MNIST data can be complex and vary slightly from one class to the other or from one image to the other. In general, the impact of noise scale on accuracy can vary based on several factors:

- 1- Amount of Noise: Increasing the noise scale 'b' can potentially disrupt the underlying patterns in the data. If the noise level is too high, it might hinder the model's ability to learn meaningful features, leading to reduced accuracy. And even observing the perturbed images with our own eyes, we can notice that with high noise, there is no way to notice a pattern anymore, because all the pixels have randoms and jumps in values so recognizing edges and jumps between pixels that correspond to the class (letter or digit) would become impossible.
- 2- Model Complexity: The effect of noise on accuracy might differ depending on the complexity of the model. More complex models might be better at handling some level of noise, while simpler models could struggle to generalize from noisy data. However, in this homework we only test a simple model.
- 3- Training Scheme: The impact of noise scale could vary based on the training hyperparameters, such as the optimizer used, learning rate, and number of epochs used during training. However, in general, the model that I chose portrays the impact of the noise, since initially the model shows good learning without noise. In some of the runs, the learning even without noise wasn't adequate (due to small value of epochs and learning rate), these runs were disregarded, as they cannot be used to compare the impact of the noise.

Finding the optimal noise scale often involves experimentation and validation. It's common to perform hyperparameter tuning, including the noise scale, to determine the value that maximizes the model's performance on validation or test data.

In some cases, higher noise levels might help prevent overfitting, leading to better generalization. This can result in fluctuations during training as the model tries to find a balance between fitting the noisy data and generalizing to unseen data. And we can see this fluctuation in Figure 11, where the training accuracy seems to decrease until $b=0.5$ and then increase again afterwards. What happens when using scale $=0.5$, is that the pixels look very random and almost hide the pattern that blends in the rest of the pixels, so it is odd to see that the training accuracy increases.

Looking at the results from run 2, we notice that the learning quality is somewhat similar when the scale (b) is between 0.03 and 0.1, as the test accuracy is similar while the validation accuracy starts decreasing between $b=0.07$ and $b=0.1$, because overfitting is less likely in these scenarios, when we add noise and acts as a form of regularization. However, given that these scales (0.03 up to 0.1 have similar test accuracy), it would be fair to conclude that raising the scale of noise till 0.1 won't affect the learning quality significantly while adding noise to the model. And looking at the perturbed image of with scale of noise $=0.1$ below, we find that indeed the noise added to the image makes it difficult to read the pattern with mere eyes, and this shows that the pixels have been perturbed enough to increase the privacy of the data, and confuse a hacker in trying to retrieve the original image. So, I think **$b=0.1$** is an optimal noise scale.



We can also notice that all scales >0.8 reduce the learning quality significantly that it becomes near impossible to learn (with the simple model that we are using), and even with more communication rounds. These cases show that the validation accuracy is stuck at a low plateau. So, any scale that is higher than 0.8 will not be appropriate for our dataset and will increase privacy on the expense of learning quality.

The test accuracy decreases somehow (not exactly) logarithmically for the cases of $b=0.1$ up to $b=0.8$. For $b=0.5$, the test accuracy is almost half of the test accuracy when using $b=0.1$, so it makes a significant drop. And with $b=0.2$, the accuracy becomes slightly less than 0.5. For this reason these values are not recommended. However, if privacy is extremely crucial and we are willing to increase computational time and improve the model, we can possibly use $b=0.2$, $b=0.3$, or $b=0.4$, for a considerable increase in privacy and a slightly more decrease in learning quality. However, with $b=0.5$, privacy doesn't seem to improve that much while the learning quality drops significantly. So, increasing the scale to $b=0.5$ is not worth it. In my opinion, privacy should be defined numerically as the amount of effort or enumeration of combinations that a hacker should test to obtain the correct original image.

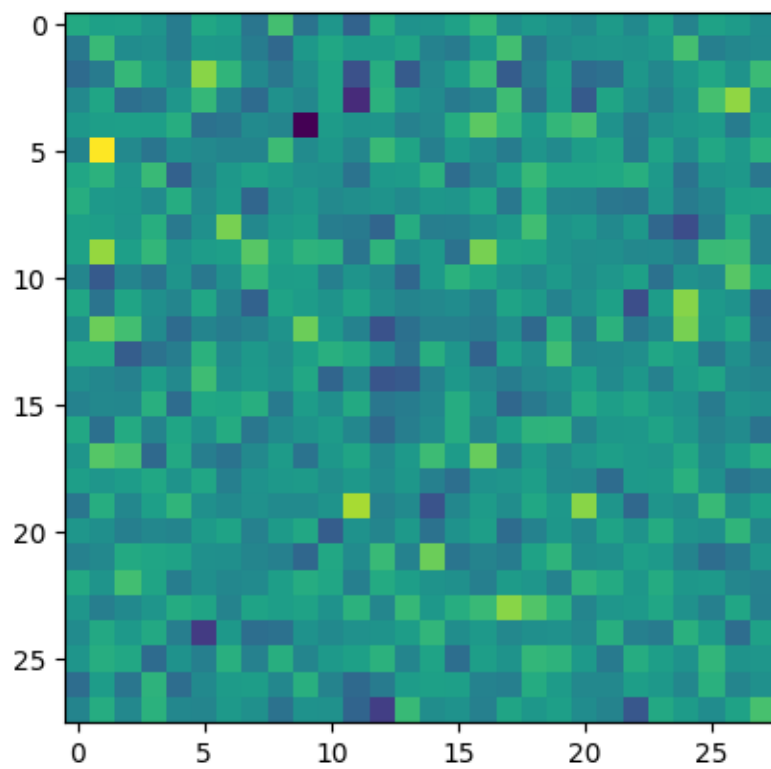


Figure 22: Noisy image with $b=0.3$

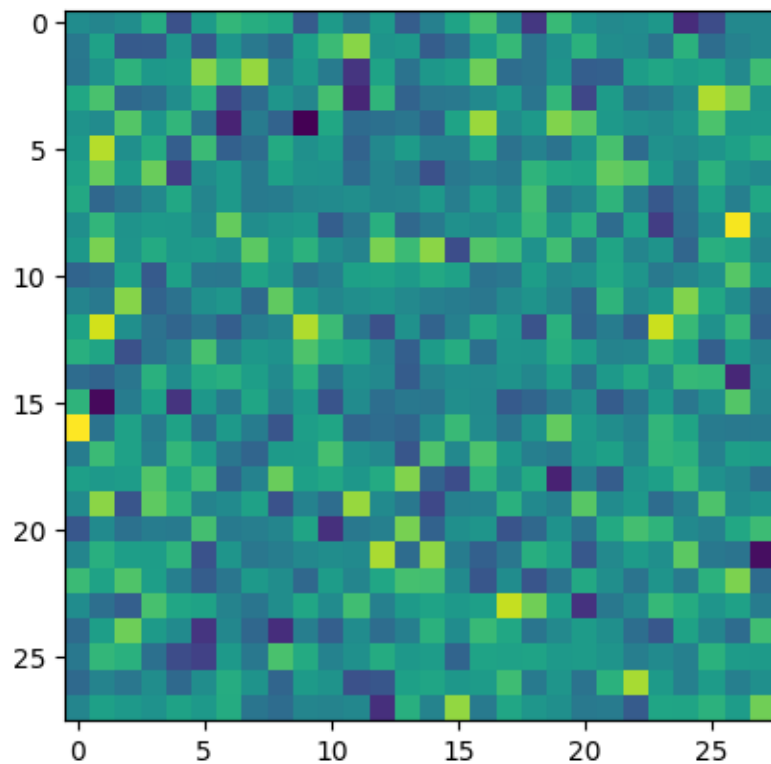
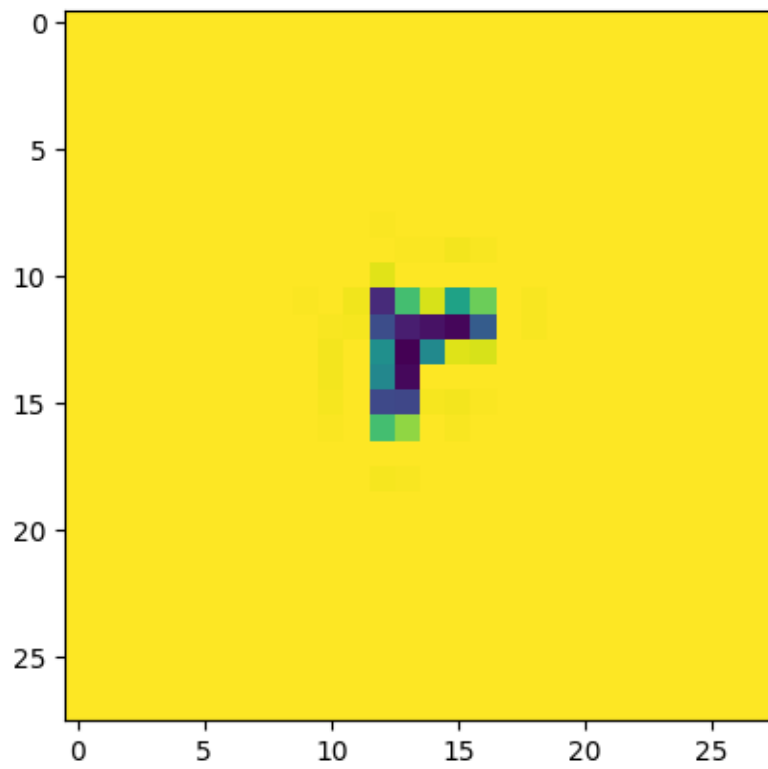


Figure 23: Noisy Image with $b=0.5$

Noise for pixels less than 1 only

I didn't end up testing this case, but I thought it might be interesting to blur only the area in proximity to the pattern rather than the whole image, to test if it improves the learning quality, so there is no need to blur out the yellow background that is the same for all images and across all clients.



Acknowledgements:

- Chatgpt
- TA